

ViDA: Lossless VideoQA Acceleration via Selective Sparse Self-Speculation with Parallel Computational Load Management

Liang Zheng[✉], Yitao Hu[✉], Chen Shen[✉], Zhixin Zhao[✉], *Graduate Student Member, IEEE*, Bowen Shi[✉], Yuhao Zhang, Laiping Zhao[✉], *Member, IEEE*, Wenxin Li[✉], and Keqiu Li[✉], *Fellow, IEEE*

Abstract—Video large language models (VideoLLMs) have significantly advanced video question answering (VideoQA) applications, which demand both low latency and high accuracy. To meet the requirements, VideoLLMs are typically deployed on GPUs for parallel acceleration. However, the massive computational load from long video contexts often makes such acceleration insufficient. Existing techniques like token pruning and speculative decoding attempt to address this challenge by altering the computational load, but often fail to balance both speed and accuracy. Sparse self-speculation mitigates these limitations via selecting a subset of tokens on a specific token budget to draft the output and then verify it using all tokens. However, existing sparse self-speculation is designed for text-based scenarios and *cannot* be directly applied to VideoQA tasks, as it fails to account for discrepancies in critical multimodal tokens and the dynamic nature of optimal token budget in VideoQA, leading to suboptimal scale and inappropriate composition of parallel computational load.

We argue that achieving both high accuracy and low latency in VideoQA tasks requires managing the computational load with awareness of these discrepancies and dynamics. To achieve this, we introduce ViDA, a selective sparse self-speculation inference system. It progressively searches token budgets by iteratively refining lower and upper bounds of search space derived from long contexts, aiming to find and allocate varying optimal budgets in real-time adaptively. Additionally, it leverages insights from discrepancies in critical multimodal tokens to perform a discrepancy-aware token selection approach for identifying critical tokens. Evaluations across various VideoQA workloads show that compared to state-of-the-art methods, ViDA preserves exact model outputs while reducing average time-per-output-token (TPOT) by 15% to 46% and average end-to-end latency by up to 28%, while decreasing the divergence from optimal token budget distribution by up to 93% and increasing the optimal critical token ratio by up to 380%.

Index Terms—VideoLLM inference, parallel computational load management, time-per-output-token, sparse self-speculation

I. INTRODUCTION

WITH the rapid development of large language models (LLMs), multimodal LLMs have become increasingly powerful and are widely applied across various fields [1], [2], [3]. Specifically, video LLMs (VideoLLMs) can perform video question answering (VideoQA) tasks based on input videos and user textual queries, generating textual responses [4], [5], [6]. For example, in intelligent video analysis, users can inquire about specific events or objects within a live

video stream and receive real-time responses [7], [8]. These applications necessitate stringent demands on usability and correctness, thereby requiring low latency and high accuracy.

To meet the low-latency demand, LLMs are usually deployed on GPU accelerators[9], [10], which leverage parallel computing to efficiently process the dense matrix operations within them[11], [12], [13]. However, VideoLLMs’ massive input contexts derived from video frames create an enormous computational load, often rendering GPU parallel acceleration alone insufficient for real-time applications.

To achieve further performance gains beyond raw hardware capability, existing systems primarily focus on altering the GPU computational load. This has led to three main techniques. The first is token pruning [14], [15], [16], [17], which reduces input length by identifying and eliminating unimportant tokens, thereby decreasing computational costs. However, it can intuitively degrade output quality, undermining the objective of correctness. The second is speculative decoding [18], [19], [20], which achieves lossless inference by utilizing a smaller and computationally cheaper draft model to generate outputs speculatively and a target model to verify them. However, since finding a proper draft model is challenging, speculative decoding typically relies on draft models that are fine-tuned with specific data, making it less generalizable and compromising its usability for VideoQA tasks.

As a combination of the above two techniques, sparse self-speculation [21] enables lossless inference without the need for a separate draft model. It adjusts the parallel computational load by leveraging token sparsity, selecting critical tokens on a token budget (*i.e.*, the number of critical tokens selected) to generate draft tokens for faster computation and then verifying them using all tokens for correctness. However, this approach is originally designed for text-based scenario and cannot be directly applied to VideoQA tasks because it uses a *uniform* strategy, which usually adopts static token budget and fixed sparsity strategies. This leads to two fundamental issues regarding the computational load: (1) Suboptimal scale, which stems from selecting the wrong number of tokens. A fixed number of critical tokens are selected without considering the dynamic nature of the optimal token budget, leading to a computational scale that is persistently mismatched with the task’s requirements. (2) Inappropriate composition, which stems from selecting the wrong set of tokens. The selection strategy lacks awareness of discrepancies in multimodal tokens, causing it to form a computational load with an

unsuitable composition that ultimately limits the acceptance rate.

In this paper, we argue that **achieving both high accuracy and low latency in VideoQA tasks requires managing the parallel computational load with awareness of the discrepancies of critical tokens and the dynamic nature of the optimal token budget inherent in VideoQA workloads.** To achieve this, we design and implement a selective sparse self-speculation inference system called ViDA, which effectively accelerates the inference process while maintaining the original model output. It accounts for these VideoQA-specific characteristics and adaptively allocates token budgets and selects critical tokens as follows.

First, ViDA introduces a novel adaptive budget allocation method to address the question of *how many tokens to select*. To adapt to the dynamics in the optimal budget, it progressively searches and allocates budgets by iteratively refining the lower and upper bounds within the token budget search space, aiming to quickly converge and allocate a proper token budget, rather than exhaustively searching the entire search space. By doing so, ViDA allocates proper token budget in real-time, thereby improving acceleration performance.

Second, ViDA proposes a novel discrepancy-aware token selection method to determine *which tokens to select* given the allocated token budget. It employs a dynamic token selection that accounts for discrepancies across modalities, iterations, layers, and requests by identifying an opportunity to reuse critical token indices and performing token selection in the prefill and verification phases. By doing so, ViDA selects proper critical tokens, thereby improving acceptance rate and enhancing acceleration performance.

ViDA is the first sparse self-speculation inference system designed for VideoQA tasks to our knowledge. It ensures both usability and correctness for VideoQA tasks, making the following contributions:

- ViDA reveals discrepancies in multimodal tokens and dynamics of the optimal budget. It provides a systematic analysis of how existing uniform sparse self-speculation, originally designed for text-based scenarios, fails to deliver high speedup for VideoQA tasks.
- ViDA designs a VideoQA-specific selective sparse self-speculation with its adaptive budget allocation and discrepancy-aware token selection to address the questions of *how many tokens to select* and *which tokens to select*, so as to select the appropriate token budget and token set. This enables effective management of the parallel computation load, thereby providing high inference speedup.
- Evaluation shows that when compared to state-of-the-art sparse self-speculation inference system, ViDA preserves exact model outputs while reducing average time-per-output-token by 15% to 46% and average end-to-end latency by up to 28%, while decreasing the divergence from optimal budget distribution by 56% to 93%, and increasing the optimal critical token ratio by 13% to 380%. An ablation study quantifies the impact of ViDA's design choices on inference acceleration for VideoQA tasks.

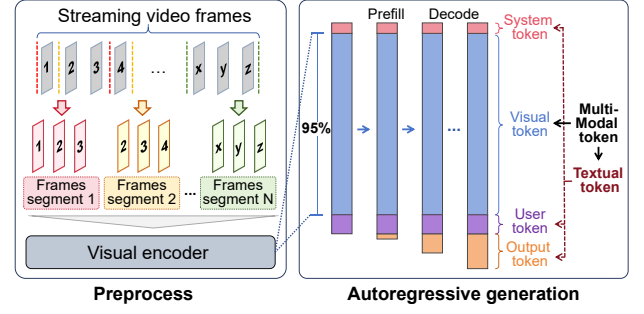


Fig. 1: Inference lifecycle of VideoQA task.

II. BACKGROUND

A. VideoLLMs inference lifecycle

In a VideoQA task, the input consists of a video stream alongside a user query, and the output is a model-generated textual response. The goal of VideoQA is to understand the visual content of video in conjunction with user query to generate accurate and relevant answers. Large language models that are specifically pretrained or fine-tuned to address VideoQA tasks are referred to as VideoLLMs [4], [5], [6].

Figure 1 shows the inference process for VideoLLMs, consisting of preprocessing and autoregressive generation. In the *preprocessing* phase, it splits the incoming video streams into smaller segments, then translates each segment into visual tokens using visual tokenizer. These visual tokens are combined with textual tokens to form an input token sequence, which is fed into VideoLLM to generate output in an autoregressive manner, comprising *prefill* and *decoding* phases. In the *prefill* phase, VideoLLM processes the input token sequence to generate the first output token in a single iteration. In the subsequent *decoding* phase, it generates one output token per iteration using the previously generated tokens. The key-value cache (KV-cache) is used to optimize the autoregressive generation by storing intermediate key-value pairs, thereby reducing redundant computations.

During the autoregressive generation, VideoLLM's token sequence consists of multimodal tokens, which are primarily divided into visual tokens and textual tokens. The textual tokens are further categorized into three types: system tokens, user tokens, and output tokens, as shown in Figure 1. One key distinction between VideoLLMs and traditional text-based LLMs lies in the inclusion of visual tokens, which often dominate the token sequence [22], [23]. This prevalence of visual tokens significantly contributes to extended inference contexts for VideoLLMs, leading to higher computational load and slower generation [24], [25], [26].

B. Existing inference acceleration methods

To address the issue of slow generation, existing inference systems have introduced the following techniques.

Token Pruning. Token pruning optimizes computations by identifying and eliminating less significant tokens. Specifically, this method first estimates the importance of input tokens, then discards [14], [15] or merges [16], [17] the unimportant tokens to shorten the inference context, aiming to reduce

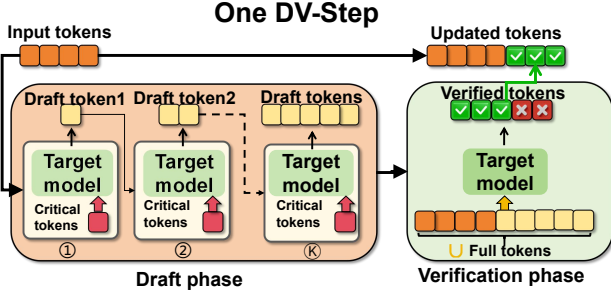


Fig. 2: Sparse self-speculation illustration.

computations. By doing so, this method reduces latency while focusing on the most relevant information. However, such a method can degrade the quality of the output, causing the output to differ from those before pruning [14], [16].

Speculative Decoding. To achieve lossless acceleration, speculative decoding has been proposed. Specifically, it accelerates the generation process by using a smaller and faster draft model to generate candidate draft tokens. Then, the target model verifies these tokens by computing their logits through a forward pass and discards the inferior ones to ensure lossless output [18], [19], [27]. We define a *DV-Step* as the process of drafting and verifying as above. The speedup is influenced by three key factors as in Equation 1: draft latency (T_{draft}), lookahead (k), which determines how many tokens are drafted in a DV-Step, and acceptance rate (α), representing the proportion of draft tokens accepted by the target model. Additionally, T_{auto} and T_{verify} represent the autoregressive decoding latency and verification latency.

$$\text{Speedup} = \frac{(k \times \alpha + 1) \times T_{\text{auto}}}{k \times T_{\text{draft}} + T_{\text{verify}}} \quad (1)$$

However, finding an appropriate draft model is challenging [28], [27]. Therefore, draft models must be fine-tuned on specific datasets, making them sensitive to data distribution changes and limiting their acceptance rates [29], [20].

Sparse Self-Speculation. As a combination of the above two techniques, sparse self-speculation leverages sparsity within the target model to serve as the draft one [21]. Specifically, it uses the target model with sparse critical tokens to draft and the full tokens to verify, as shown in Figure 2. By doing so, it overcomes the output degradation caused by token pruning and does not require finding a dedicated draft model like speculative decoding. Two key factors affect the acceleration performance: the token budget, which determines how many critical tokens to select, and the selection strategy, which determines which tokens to choose.

Since the current sparse self-speculation framework was originally designed for text generation scenarios and usually adopts static token budgets and fixed sparsity strategies, we define this paradigm as *uniform* sparse self-speculation. We argue that such uniform paradigm cannot fundamentally resolve the inefficiency in VideoQA tasks, as the uniform design fails to align with the unique characteristics of multimodal tokens, which we will discuss next.

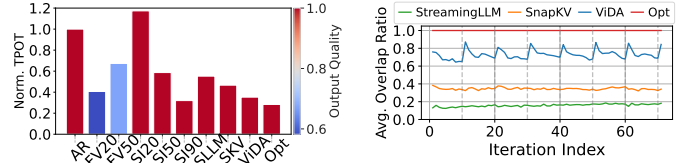


Fig. 3: Performance comparison of current methods.

Fig. 4: Overlap ratios with optimal token indices.

III. MOTIVATION

A. Limitation of current methods on VideoQA

To validate the limitations of the three aforementioned inference acceleration methods in VideoQA, we evaluate their representative approaches individually. For token pruning, we conduct experiments on FastV [14] (abbreviated as FV) by keeping only 20% and 50% of the tokens with the largest summed attention weights. For speculative decoding, we analyze SpecInfer [18] (abbreviated as SI) by simulating the speedup of draft models at various acceptance rates. For sparse self-speculation, we examine MagicDec [21], which employs two sparse mechanisms (abbreviated as SLLM and SKV). We set the token budgets to 512 for SLLM and 2049 for SKV, as described in the original paper [21]. Additionally, we report the theoretical optimum (denoted as Opt), which is obtained via an exhaustive search. The lookahead parameter k is set to 10 for methods that require it.

We conduct experiments using the LongVA-qwen2-7B model on the MLVU-summary dataset¹, as described in §V-A. To evaluate speedup performance, we use normalized time per output token (nTPOT), which is calculated by dividing each TPOT value by the autoregressive (AR) TPOT. To assess the impact of acceleration on output quality, we measure the output consistency with the baseline AR method using Longest Common Subsequence (LCS) similarity. A proper acceleration method should have small nTPOT, alongside high LCS similarity.

As shown in Figure 3, FastV achieves nTPOT of 0.41 and 0.67, alongside LCS similarity of 0.58 and 0.67 with 20% and 50% critical tokens. This indicates that token pruning results in substantial sacrifice in output quality due to information losses. Meanwhile, SpecInfer exhibits huge performance fluctuation with nTPOT of 0.32, 0.59 and 1.17 under simulated acceptance rate of 90%, 50% and 20%. As discussed, it is challenging to find a proper draft model. Recent works [29], [20] show that the acceptance rate is typically below 50%, leading to an nTPOT no less than 0.59.

Besides, although MagicDec surpasses FastV and SpecInfer with nTPOT of 0.55 and 0.47 under various sparse mechanisms, it still falls far short of the theoretical optimum with nTPOT of 0.28. This substantial gap indicates that directly applying solutions from text-based scenarios to VideoQA tasks fails to resolve the issue of slow generation. To further understand the limitations of text-based method, we conduct

¹Existing acceleration methods show similar limitations on various combinations of model and dataset. We omitted them for brevity.

a thorough analysis of it from two key aspects: which tokens to select and how many tokens to select.

B. Text-based method selects wrong tokens

The existing text-based sparse self-speculation [21] primarily employs two types of sparse strategies: StreamingLLM [30], which selects critical tokens from the beginning and end of the sequence, and SnapKV [31], which selects different critical tokens for each request and reuses these tokens in subsequent iterations. To assess their impact on end-to-end acceleration, we measure the overlap between the selected critical tokens and the theoretical optimal set, which is derived at each iteration by computing attention weights over full sequence. A higher overlap implies that the chosen tokens are closer to the optimal and thereby more important.

Figure 4 shows that in the VideoQA setting the overlap ratio of both sparsification strategies stays below 39%. This suggests that current sparse approaches tend to select suboptimal token sets for VideoQA.

To further explore why current method fails to select appropriate critical tokens, we analyzed the multimodal attention weights generated in VideoQA to determine the patterns of critical tokens. As shown in Figure 5a, the causal nature of attention ensures each token can only attend to itself and preceding tokens, resulting in a lower triangular matrix of attention weights. With each new token decoded, a new row is appended to the matrix. The element at row i and column j represents importance of the j -th token for the i -th token. Our analysis found four sparse patterns as follows.

Observation #1: Modality Discrepancy. Sparsity patterns in multimodal tokens differ significantly from text-only data because each modality has unique token distribution characteristics. Specifically, textual tokens, including system tokens, user tokens and output tokens, show high heat values, indicating that they are relatively important. The visual tokens in the middle alternate between high and low heat values, reflecting their sparsity. This pattern highlights the discrepancy between textual and visual modalities.

Observation #2: Iteration Discrepancy. In the region of the output tokens in Figure 5b, each row corresponds to a successive iteration. As iteration advances, the critical visual tokens emerge on different columns, revealing that the critical visual tokens change over time instead of remaining fixed. This row-wise shift highlights a clear iteration discrepancy in the distribution of critical visual tokens.

Observation #3: Layer Discrepancy. As in Figure 5b, the positions of critical visual tokens shift across layers within each request, indicating inter-layer discrepancies.

Observation #4: Request Discrepancy. As in Figure 5b, the positions of critical visual tokens vary between requests within each layer, indicating inter-request discrepancies.

The four discrepancies above highlight the limitations of current text-based methods for VideoQA. The StreamingLLM strategy selects tokens based on fixed positions, which fails to adapt to the iteration, layer and request discrepancy (ob 2-4 ✕). The SnapKV strategy, although it selects different positions

for each request, keeps the same token selection throughout subsequent iterations, ignoring the iteration discrepancy (ob 2 ✕). Both methods treat all tokens as part of a single modality, missing the modality discrepancy (ob 1 ✕). These strategies of current method lead to poor token selection in Figure 4.

C. Text-based method selects wrong budget

The existing text-based sparse self-speculation [21] utilizes a fixed token budget, determined through empirical settings. However, we observed that these budget configurations are suboptimal for VideoQA tasks. Figure 6 shows the speedup distribution of various token budgets compared to the autoregressive baseline, derived from brute force across randomly sampled DV-Steps. An analysis of it reveals a key observation.

Observation #5: DV-Step Dependency. Figure 6 shows that within a single DV-Step, the speedup achieved by different token budgets varies significantly, with the maximum difference reaching up to 4.3 times (in DV-Step 2), highlighting the substantial impact of budget selection on performance. Additionally, Figure 6 also reveals that the budget yielding the highest speedup varies dynamically across different DV-Steps, with the optimal budgets identified in the figure occurring at 3000, 3500, 5500 and 7500.

To further understand the DV-Step dependency of the optimal budget, we analyzed the trade-off between the acceptance rate (α) and draft latency (T_{draft}) in terms of the token budget: increasing the budget improves α but also prolongs T_{draft} . The overall speedup improves only if the gain in α outweighs the additional cost in T_{draft} .

Importantly, for a fixed budget, T_{draft} is determined solely by the model architecture and hardware, remaining constant across DV-Steps. In contrast, α varies dynamically. For example, as shown in Figure 7, α under the token budget of 5000 changes over the 30 sampled DV-Steps. Since α exhibits this variation while T_{draft} does not, the trade-off shifts dynamically among DV-Steps, which explains why the optimal budget varies across DV-Steps in Figure 6.

Based on observation above, current fixed budget strategy can only achieve suboptimal speedup in VideoQA tasks.

D. Implications

The analysis above highlights that existing *uniform* paradigm of sparse self-speculation designed for text-based tasks cannot handle VideoQA tasks. To overcome this limitation, ViDA must adopt a *selective* strategy, addressing two key questions: (1) *Which tokens to select?* ViDA needs to be aware of discrepancies in tokens of multimodal tasks and dynamically select critical tokens accordingly. (2) *How many tokens to select?* ViDA should adaptively allocate budgets for each DV-Step while considering trade-off between α and T_{draft} .

However, designing such a system presents several challenges: (1) *Identifying critical tokens in advance:* Addressing the first question requires identifying dynamic critical multimodal tokens in VideoQA tasks before computing attention. This is challenging because selecting these tokens to reduce computation relies on full attention weights, which are calculated between all token pairs. Such a requirement

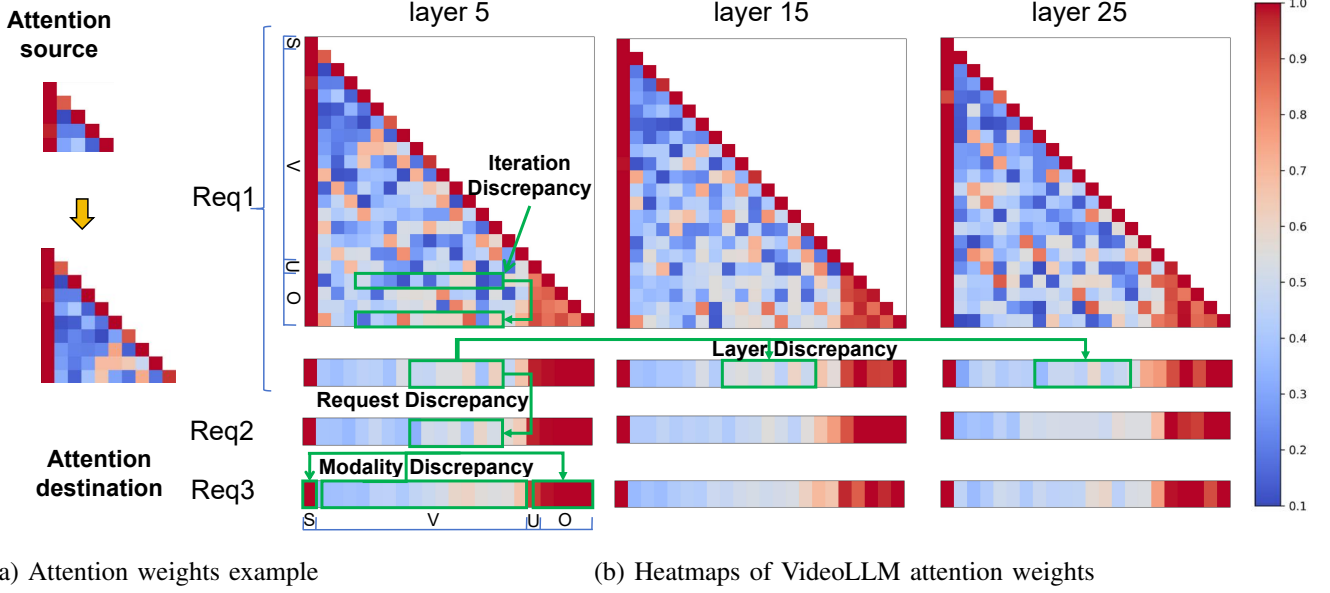


Fig. 5: Attention heatmaps of VideoLLM model, computed from downsampled tokens. The horizontal bar beneath each heatmap shows the column-wise average of the lower-triangular attention weights. S, V, U and O denote the corresponding system, visual, user and output tokens, respectively.

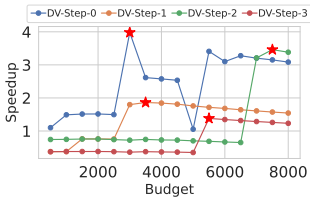


Fig. 6: Speedup of various DV-Steps under varying token budgets.

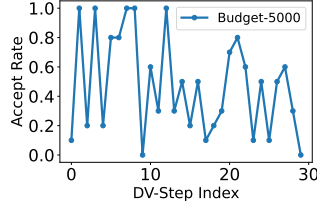


Fig. 7: Acceptance rate α with token budget 5k under varying DV-Steps.

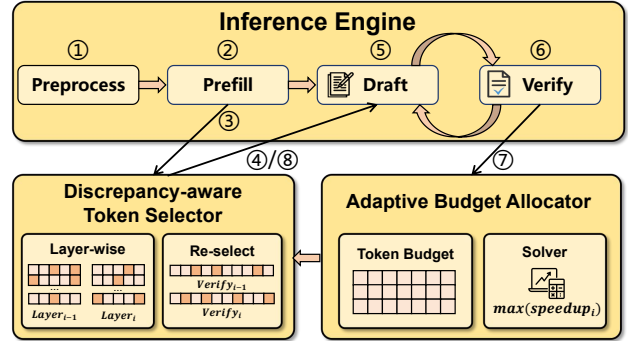


Fig. 8: ViDA overview.

contradicts the goal of reducing the computational cost of attention, resulting in a circular dependency. (2) *Identifying the optimal budget in real-time*: Addressing the second question requires adaptively selecting the optimal budget for each DV-Step in real-time. This is challenging due to the vast search space of possible budgets, especially given the long context in VideoQA. Next, we will introduce how ViDA addresses these two critical challenges through adaptive budget allocator and discrepancy-aware token selector.

IV. DESIGN

We start with an overview of ViDA (§IV-A), followed by its two key designs, adaptive token budget allocator (§IV-B) and discrepancy-aware token selector (§IV-C), both of which are critical to guaranteeing high acceleration performance.

A. System overview

We propose ViDA, an efficient VideoLLM inference system that dynamically selects critical tokens with proper token budget for *selective* sparse self-speculation. As shown in Figure 8, ViDA contains three key components: (1) Inference engine,

which controls the preprocessing and sparse self-speculation process, including prefill and loop of draft and verification. (2) Adaptive budget allocator, which uses progressive search to allocate appropriate token budget tailored to each DV-Step. (3) Discrepancy-aware token selector, which dynamically selects critical tokens on specific token budgets and constructs their KV-cache for computations during the draft phase.

ViDA first performs necessary initialization steps, including allocating KV pool and loading model weights. Upon receiving a request, ViDA preprocesses video streams alongside user queries into input token sequence ①. It then executes the prefill phase ②, during which the discrepancy-aware token selector identifies initial critical tokens with initial token budget ③ and constructs their sparse KV-cache, which is then used for initial draft phase ④. When entering the draft-verification loop, the system first goes through the draft phase, using the sparse KV-cache for computation to generate k draft tokens ⑤. It then uses the full KV-cache to verify these draft tokens and records the attention weights for re-selection during verification phase ⑥. Next, the adaptive token budget allocator searches and



Fig. 9: Draft latency under varying token budget.

allocates token budget based on a progressive search algorithm ⑦. Then, the system re-selects critical tokens and prepares the KV-cache corresponding to them for the subsequent draft phase ⑧.

Because token selection is constrained by a specific token budget, we will first describe the budget allocation process and then outline the token selection process.

B. Adaptive token budget allocator

Insight and approach. Based on the observation in §III-C, ViDA needs to adopt an adaptive token budget allocation method to balance draft latency T_{draft} and acceptance rate α . However, for VideoQA tasks, the token budget search space is extremely large, making the complexity of an exhaustive search prohibitive. One feasible approach is to discretize the search space. Nevertheless, even after discretization, the scale of search space remains immense, and relying on a brute-force search still fails to meet real-time requirements.

Fortunately, we developed an iterative heuristic that can progressively find a near-optimal budget. Instead of exhaustively searching the entire space, this method operates within the discretized search space by alternately narrowing the lower and upper bounds and tracking the traversed budgets, as illustrated in Figure 10. Experiment in §V-C demonstrates that this progressive search method can achieve over 94% of the speedup obtained when using the optimal budget. Compared to a discretized brute-force search, it saves more than 91% of complexity, while obtaining the same reachable optimal budget (B_{ReaOpt}).

The iterative heuristic has three components: (1) Inflection-based lower bound boosting: This determines the initial lower bound by analyzing budget inflection point and incrementally increases the lower bound during the search process. (2) AccDL-constrained upper bound tightening: This is used to tighten the upper bound based on current lower bound. (3) Candidate budget tracker: This records the speedup of each searched token budget, ensuring that B_{ReaOpt} is not missed. Next, we describe these three components in details.

Inflection-based lower bound boosting has two main phases to derive token budget lower bound B_{lb} as follows.

Initial lower bound determination. First, we analyzed the relationship between token budget B and draft latency T_{draft} , identifying an inflection point as shown in Figure 9a ($B_{\text{inflect}}=3k$). Before this inflection point, T_{draft} remains largely constant, while after it, T_{draft} begins to increase. To further understand that, it is essential to examine the composition of T_{draft} . It comprises two parts: GPU launch latency T_{laun} and GPU execution latency T_{exec} . These two parts can overlap each other, so we have $T_{\text{draft}} = \max(T_{\text{laun}}, T_{\text{exec}})$. Before the inflection point, T_{draft} is largely determined by the launch latency, which is relatively fixed, as shown in Figure 9b.

As the token budget increases, the draft sequence length extends, leading to an increase in the number of attention tiles, as depicted in Figure 9c. Attention tiling is the fundamental unit of parallel computation on GPU, with each tile responsible for computing a portion of the attention mechanism. When the token budget is small, the number of tiles remains within GPU’s parallel processing capacity, allowing GPU execution latency to stay at a stable level. However, as the budget further increases, the number of tiles eventually exceeds GPU’s parallel processing capability, forcing certain attention computations to be serialized, leading to increase in both attention computation latency (Figure 9d) and overall GPU execution latency (Figure 9b). After the inflection point (e.g., $B_{\text{inflect}}=3k$), GPU execution latency exceeds GPU launch latency, T_{draft} is then primarily determined by GPU execution latency and grows along with it.

For $B \in [0, B_{\text{inflect}}]$, the draft latency under any B is nearly constant, while $B = B_{\text{inflect}}$ has the highest acceptance rate since it encodes more information. Namely, the inflection point is a Pareto optimal for $B \in [0, B_{\text{inflect}}]$ in terms of inference acceleration. Thereby, we select B_{inflect} as the initial lower bound $B_{\text{lb}}^{\text{init}} = B_{\text{inflect}}$.

Iterative lower bound increment. To ensure the search proceeds iteratively and can traverse all potential B_{ReaOpt} , we proactively increase the lower bound by an amount equal to $1/n$ of the total context length (defined as search space size L) after each upper bound reduction, as illustrated in Figure 10. Here, n is a hyperparameter that controls the granularity of search space discretization. We empirically select $n = 64$ and verify its robustness in §V-D. Leveraging this discretization, we determine the exact value of B_{inflect} through an offline profiling step prior to inference. Starting from a minimal token budget, we iteratively increase it by the step size L/n . This process stops once the measured draft latency strictly exceeds the initial draft latency, as detailed in Algorithm 1.

AccDL-constrained upper bound tightening. Even though we can progressively raise the lower bound through initial determination and iterative increments, a large search space still remains for the upper bound. This is because the trade-off mentioned in §III-C indicates that the relationship between speedup and token budget is not always positively correlated.

Fortunately, based on our analysis, we derive an accept rate and draft latency (AccDL) constraint theorem showing that an upper bound exists for a better token budget, and subsequently propose a method that combines offline and online profiling to determine this upper bound.

Algorithm 1: Offline Inflection Budget Profiling

Input: Search space size L , search space split factor n
Output: Inflection budget B_{inflect}

// Define step size and initialize inflection budget
 $\Delta B \leftarrow \frac{L}{n}$;
 $B_{\text{inflect}} \leftarrow \Delta B$;
 // Record baseline launch latency at minimum budget
 $T_{\text{base}} \leftarrow \text{PROFILEDRAFTLATENCY}(B_{\text{inflect}})$;
 // Step forward and profile iteratively
while $\text{PROFILEDRAFTLATENCY}(B_{\text{inflect}} + \Delta B) \leq T_{\text{base}}$ **do**
 $B_{\text{inflect}} \leftarrow B_{\text{inflect}} + \Delta B$;
 // Return the exact point before execution latency increased
return B_{inflect} ;

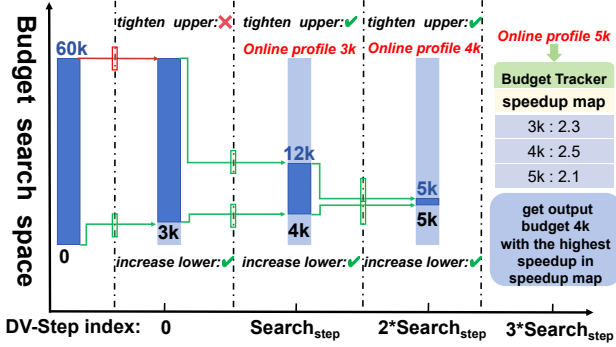


Fig. 10: Progressive search example.

Theorem 1 (AccDL constraint). Given a budget lower bound B_{lb} , there exists an upper bound B_{ub} such that any budget B_{higher} satisfying $S_{\text{higher}} \geq S_{\text{lb}}$ will only be found within the interval $[B_{\text{lb}}^{\text{init}}, B_{\text{ub}}]$.

Proof. During inference, the draft phase uses a sparse KV-cache determined by the budget B , while the verification phase strictly utilizes the full KV-cache. Consequently, the verification latency T_{verify} depends only on the total sequence length and the draft token count k , remaining independent of the draft budget B . For a given B_{lb} , let $T_{\text{draft,lb}}$ be its draft latency, α_{lb} its acceptance rate, and S_{lb} its resulting speedup. Similarly, for budget B_{higher} mentioned in the theorem, let $T_{\text{draft,higher}}$, α_{higher} , and S_{higher} denote its corresponding draft latency, acceptance rate, and speedup, respectively.

Given Equation 1, we rewrite condition $S_{\text{higher}} > S_{\text{lb}}$ as:

$$\frac{(k\alpha_{\text{higher}} + 1)T_{\text{auto}}}{kT_{\text{draft,higher}} + T_{\text{verify}}} > \frac{(k\alpha_{\text{lb}} + 1)T_{\text{auto}}}{kT_{\text{draft,lb}} + T_{\text{verify}}}$$

After simplification, we obtain:

$$T_{\text{draft,higher}} < \frac{\alpha_{\text{higher}}(kT_{\text{draft,lb}} + T_{\text{verify}}) + T_{\text{draft,lb}} - \alpha_{\text{lb}}T_{\text{verify}}}{k\alpha_{\text{lb}} + 1}$$

Since $\alpha_{\text{higher}} \leq 1$ and $(kT_{\text{draft,lb}} + T_{\text{verify}})$ is positive, we have $\alpha_{\text{higher}}(kT_{\text{draft,lb}} + T_{\text{verify}}) \leq (kT_{\text{draft,lb}} + T_{\text{verify}})$. Applying this relaxation yields the strict inequality:

$$T_{\text{draft,higher}} \leq \frac{(k+1)T_{\text{draft,lb}} + (1 - \alpha_{\text{lb}})T_{\text{verify}}}{k\alpha_{\text{lb}} + 1} \triangleq T_{\text{draft,ub}}$$

where we define the right side of inequality as $T_{\text{draft,ub}}$. Thus, any B_{higher} yielding $S_{\text{higher}} > S_{\text{lb}}$ must satisfy $T_{\text{draft,higher}} \leq$

Algorithm 2: Token Budget Progressive Search

Input: Inflection budget B_{inflect} , search space size L , search space split factor n , search steps $\text{step}_{\text{search}}$, draft token count k , verification latency T_{verify} , autoregressive latency T_{auto}
Output: Optimal token budget B_{output}

// Initialize budget candidate tracker
 $B_{\text{lb}} \leftarrow B_{\text{inflect}}$, $\text{Speedup}_{\text{max}} \leftarrow 0$, $B_{\text{ub}} \leftarrow \infty$;

while $B_{\text{lb}} \leq B_{\text{ub}}$ **do**
 // Online profiling for actual acceptance rate
 $\alpha_{\text{lb}} \leftarrow \text{PROFILEACCEPTANCERATE}(B_{\text{lb}}, \text{step}_{\text{search}})$;
 // Retrieve offline profiled draft latency
 $T_{\text{draft,lb}} \leftarrow \text{GETOFFLINELATENCY}(B_{\text{lb}})$;
 // Calculate current speedup
 $\text{Speedup}_{\text{lb}} \leftarrow \frac{(k\alpha_{\text{lb}} + 1)T_{\text{auto}}}{kT_{\text{draft,lb}} + T_{\text{verify}}}$;
 // Update candidate budget tracker
if $\text{Speedup}_{\text{lb}} > \text{Speedup}_{\text{max}}$ **then**
 $\text{Speedup}_{\text{max}} \leftarrow \text{Speedup}_{\text{lb}}$, $B_{\text{output}} \leftarrow B_{\text{lb}}$;
 // Apply AccDL constrained upper bound tightening
 $T_{\text{draft,ub}} \leftarrow \frac{(k+1)T_{\text{draft,lb}} + (1 - \alpha_{\text{lb}})T_{\text{verify}}}{k\alpha_{\text{lb}} + 1}$;
 $B_{\text{ub}} \leftarrow \text{MAPLATENCYTOBUDGET}(T_{\text{draft,ub}})$;
 // Perform iterative lower bound increment
 $B_{\text{lb}} \leftarrow B_{\text{lb}} + \frac{L}{n}$;

// Return the best performing budget
return B_{output} ;

$T_{\text{draft,ub}}$. Since draft latency T_{draft} is non-decreasing with token budget B , we have $B_{\text{higher}} \leq B_{\text{ub}}$. We have $B_{\text{lb}}^{\text{init}} \leq B_{\text{higher}}$ by definition. Thereby, we prove $B_{\text{higher}} \in [B_{\text{lb}}, B_{\text{ub}}]$. $\square$

Given Theorem 1, we determine B_{ub} in two steps.

In the first step, we calculate B_{ub} 's corresponding draft latency $T_{\text{draft,ub}}$. Given Equation 1, draft latency $T_{\text{draft,ub}}$ depends on four parameters: k , $T_{\text{draft,lb}}$, T_{verify} and α_{lb} . Among them, k is predetermined, while $T_{\text{draft,lb}}$ and T_{verify} can be obtained via offline profiling. Meanwhile, as discussed in §III-C, α_{lb} cannot be determined in advance, since it is workload dependent. Thereby, we use online profiling to derive α_{lb} instead. As shown in Figure 10, once a B_{lb} is selected, we execute a specific number of DV-Steps (defined as $\text{step}_{\text{search}}$) using token budget B_{lb} . The average α observed over these $\text{step}_{\text{search}}$ DV-Steps is then used as the value for α_{lb} .

In the second step, we derive B_{ub} from the value of $T_{\text{draft,ub}}$. With α_{lb} determined online, alongside aforementioned k , $T_{\text{draft,lb}}$ and T_{verify} , we can calculate draft latency $T_{\text{draft,ub}}$ using Equation 1. We then derive its corresponding token budget B_{ub} , following the 1-to-1 correspondence between draft latency and token budget as in Figure 9a.

Candidate budget tracker. During the progressive search, the reachable² optimal performance might be achieved by intermediate B_{lb} rather than the final converged budget, due to iterative lower bound increment. To avoid missing this reachable optimal budget, we use candidate budget tracker to record the speedup of each evaluated B_{lb} . Upon convergence, the tracker selects the budget corresponding to the highest record speedup as the output token budget B_{output} .

²The global optimal is unreachable due to complexity as in §V-D.

Due to DV-step dependency (§III-C), B_{output} 's performance will gradually become suboptimal. To mitigate this, ViDA periodically re-initiates the search. A new progressive search round is triggered after using an identified B_{output} for a predefined number of DV-Steps ($step_{\text{apply}}$). The effectiveness of this re-search mechanism is closely tied to the ratio of $step_{\text{search}}$ to $step_{\text{apply}}$. We empirically set the ratio as 5 : 150 and verify its robustness in §V-D. The complete workflow of our progressive search, integrating the aforementioned lower bound boosting, upper bound tightening, and candidate budget tracker, is summarized in Algorithm 2.

C. Discrepancy-aware token selector

Insight and Approach. Based on the observations in §III-B, sparsity patterns in VideoQA exhibit discrepancies across four aspects: modality, iteration, layer and request. Consequently, even when an optimal token budget is given, the optimal token set varies. To select an appropriate critical token set, ViDA proposes a discrepancy-aware token selection strategy. Specifically, for modality discrepancy, ViDA retains all textual tokens and only sparsifies visual tokens; For discrepancies across layers and requests, ViDA dynamically selects distinct critical visual tokens independently for each layer and each request; For iteration discrepancy, which appears as varying critical visual tokens during the draft phase, ViDA dynamically changes the critical visual tokens throughout the draft phase.

However, as highlighted in §III-D, the dynamic token selection introduces a circular dependency: the pre-selection of critical tokens aimed at reducing attention computation paradoxically depends on the results of a full attention computation. To overcome this challenge and facilitate dynamic selection, ViDA takes two steps. First, it conducts a deeper analysis of the critical token overlap ratio across iterations. This identifies opportunities for reusing critical visual token indices and restricts token selection to only the prefill and verification stages. Second, ViDA introduces a visual token selection method that leverages attention weights to identify suitable token sets during both prefill and verification stages; this method also corrects for the potential inclusion of rejected tokens in the computation during the re-selection of the verification stage.

Opportunity to reuse critical token indices. Though critical tokens change between iterations (§III-B), we find that in adjacent iterations, critical tokens exhibit substantial similarity. We define the similarity as the overlap rate of the top-3000 most critical visual tokens. Figure 11 shows that the similarity is high for nearby iterations (e.g., similarity > 0.75 when gap < 10) and decreases as iteration gap widens.

Attention-guided critical visual token selection. The above observation identifies an opportunity to reuse critical token indices within small iteration gap. Specifically, it allows us to perform full computation in one iteration to get critical token indices and then reuse the indices in subsequent adjacent iterations, so that subsequent iterations can get critical tokens by inheriting the previous selection results instead of through full attention computation.

Notably, we find that sparse self-speculation is naturally suited for this pattern for two key reasons. First, because the

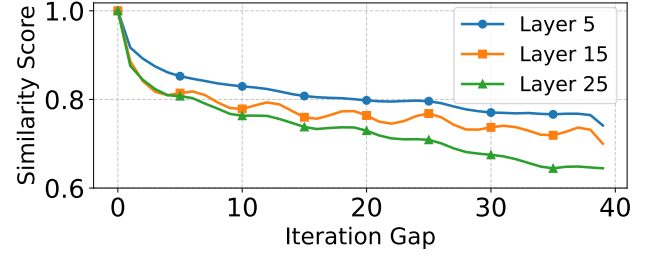


Fig. 11: Critical visual tokens similarity.

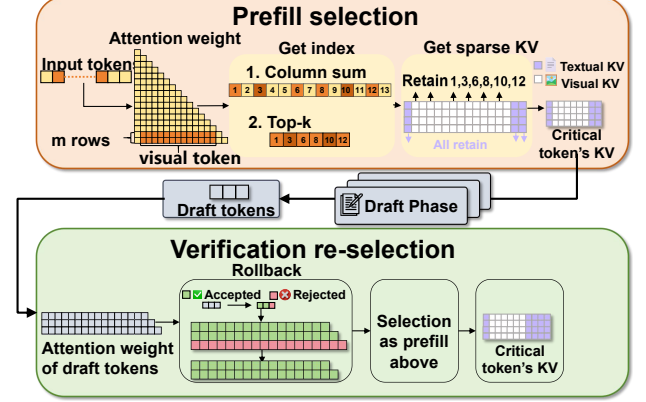


Fig. 12: Attention-guided token selection illustration.

prefill stage and verification stage inherently need to undergo full computation, only requiring minimum extra overhead³ for critical token identification. Second, token selection in verification phase enables dynamic changes in critical visual tokens across different DV-Steps, aligning with the iteration-discrepancy. Therefore, we can leverage the prefill stage for initial selection and the verification stage for dynamic re-selection, while reusing the selection results during the draft phase. Due to layer discrepancy and request discrepancy, we perform token selection and re-selection independently for each request and each layer. Next, we describe how to perform the selection and re-selection as in Figure 12.

Selection in prefill phase. To perform token selection in prefill phase, we should first measure the importance of each visual token and then select the most important ones.

As described in §III-B, the attention weight reflects the importance of different tokens, so we can leverage this property to measure visual token importance. To implement this, we first calculate the attention weight matrix $\mathbf{A}$. Then, we perform a column sum on the region of matrix $\mathbf{A}$ corresponding to the visual columns and the last m rows, as follows:

$$V_j = \sum_{i=L-m+1}^L \mathbf{A}_{i,(N_s+j)} \quad \text{for } j \in \{1, \dots, N_v\} \quad (2)$$

Here, j indexes the visual tokens from 1 to N_v . The column index in $\mathbf{A}$ for the j -th visual token is $N_s + j$, where N_s is the number of system tokens, and L is the total sequence length.

The motivation for summing these m rows stems from the autoregressive nature of the generation process, where the current generation step is typically most influenced by the latest few tokens. By aggregating the attention weights from

³We will quantify the overhead in §V-D.

Algorithm 3: Discrepancy-Aware Token Selection

Input: Attention matrix A , total sequence length L , system token count N_s , visual token count N_v , lookback window m , output budget B_{output} , textual token indices I_{text}

Output: Final critical token indices I_{final}

// Initialize importance scores for all visual tokens
 $V \leftarrow$ Array of size N_v initialized to 0;

// Calculate visual token importance using recent m tokens
for $j \leftarrow 1$ **to** N_v **do**
 $V_j \leftarrow \sum_{i=L-m+1}^L A_{i, N_s+j}$;

// Select critical visual tokens based on budget
 $I_{\text{crit}} \leftarrow \arg \text{topk}_{j \in \{1, \dots, N_v\}}(V_j, B_{\text{output}})$;

// Merge selected visual tokens with textual tokens (modality discrepancy)
 $I_{\text{final}} \leftarrow I_{\text{crit}} \cup I_{\text{text}}$;

// Return indices to construct sparse KV-cache
return I_{final} ;

the m most recently generated tokens, this heuristic helps approximate which visual features are important, reducing the interference of irrelevant attention weights from older tokens. We empirically set $m = 16$ and verify its robustness in §V-D. Following the above steps, we can derive the importance score for each visual token. Then, using these importance scores V_j , the indices I_{crit} identifying the B_{output} ⁴ most critical visual tokens can be determined as follows:

$$I_{\text{crit}} = \arg \text{topk}_{j \in \{1, \dots, N_v\}}(V_j, B_{\text{output}}) \quad (3)$$

The complete process of this discrepancy-aware token selection is summarized in Algorithm 3.

Re-selection in verification phase. The re-selection process in the verification phase is similar to that in the prefill phase. However, a potential risk specific to applying this method during verification is that the draft tokens used for the selection computation might include tokens that are ultimately not accepted (the gray tokens in Figure 12), which can lead to an inaccurate visual token importance assessment.

To ensure the selection computation is not affected by rejected tokens, we postpone the token selection process until the set of accepted draft tokens and the set of rejected ones are determined, and then conduct it using only accepted tokens by rolling back matrix A to exclusively hold the attention weights of accepted tokens, as shown in Figure 12. This rollback and re-selection procedure is detailed in Algorithm 4.

With the aforementioned methods, critical visual tokens can be selected in both the prefill and verification stages. Then, these selected critical visual tokens are combined with all textual tokens to form the final set of critical tokens. We can further select the KV-cache corresponding to these critical tokens and use it for the draft phase.

Algorithm 4: Rollback and Token Re-selection

Input: Verification attention matrix A , original sequence length L_{old} , draft token count k , acceptance results R

Output: Re-selected token indices I_{final}

// Count accepted draft tokens
 $k_{\text{acc}} \leftarrow \text{COUNTACCEPTEDTOKENS}(R)$;

// Calculate clean sequence length
 $L_{\text{clean}} \leftarrow L_{\text{old}} + k_{\text{acc}}$;

// Truncate matrix to exclude rejected tokens
 $A_{\text{clean}} \leftarrow A[1 : L_{\text{clean}}, 1 : L_{\text{clean}}]$;

// Re-select tokens using clean matrix via Algorithm 3
 $I_{\text{final}} \leftarrow$
 $\text{DISCREPANCY-AWARE-TOKEN-SELECTION}(A_{\text{clean}}, L_{\text{clean}}, \dots)$;

return I_{final} ;

V. EVALUATION

A. Experimental Setup

Implementation. We implemented ViDA (detailed in §IV) using approximately 10k lines of Python code with PyTorch. It integrates FlashAttention and an efficient KV-cache management strategy to enable sparse self-speculation for VideoLLMs. To prevent memory fragmentation, we maintain a standard dense KV-cache and track critical tokens using sparse indices. During attention computation, the system uses these indices for indirect addressing to directly fetch required states. This avoids complex sparse memory allocators and bounds persistent memory overhead.

Hardware Setup. We evaluated ViDA on a server equipped with one NVIDIA A800-80GB GPU and Intel Xeon-Gold-5320 CPU, with 540GB host memory.

Model and system configuration. We evaluate three mainstream VideoLLMs: LongVA-qwen2-7B [4], LongVILA-llama3-8B [5] and LLaVA-Onevision-1.5-8B [32]. These three models are capable of handling long video inputs, excelling in understanding and answering user queries in complex video scenarios. In our comparative and ablation experiments, we set lookahead $k = 10$ across all experiments. Additionally, we performed a sensitivity analysis of k in §V-D to evaluate the acceleration performance under different values of k .

Workload. To evaluate acceleration performance, we selected three key VideoQA tasks: video summarization, temporal question answering and action reasoning. These tasks are foundational to numerous VideoQA applications and present challenges in video comprehension, temporal reasoning, and action understanding.

- **MLVU-Summary (abbreviated as MLVU):** A benchmark focusing on generating structured summaries from complex, long-duration videos, requiring the identification of key information throughout the video [33].
- **Video-ChatGPT Temporal QA (abbreviated as TQA):** A benchmark focusing on answering questions about specific video segments and testing VideoLLMs' ability to reason about temporal events [34].
- **VideoMME Action Reason (abbreviated as VMME):** A benchmark from the Multi-Modal Evaluation (MME) dataset focusing on analyzing the causes, intents, and

⁴ B_{output} is the output token budget derived by ViDA as in §IV-B.

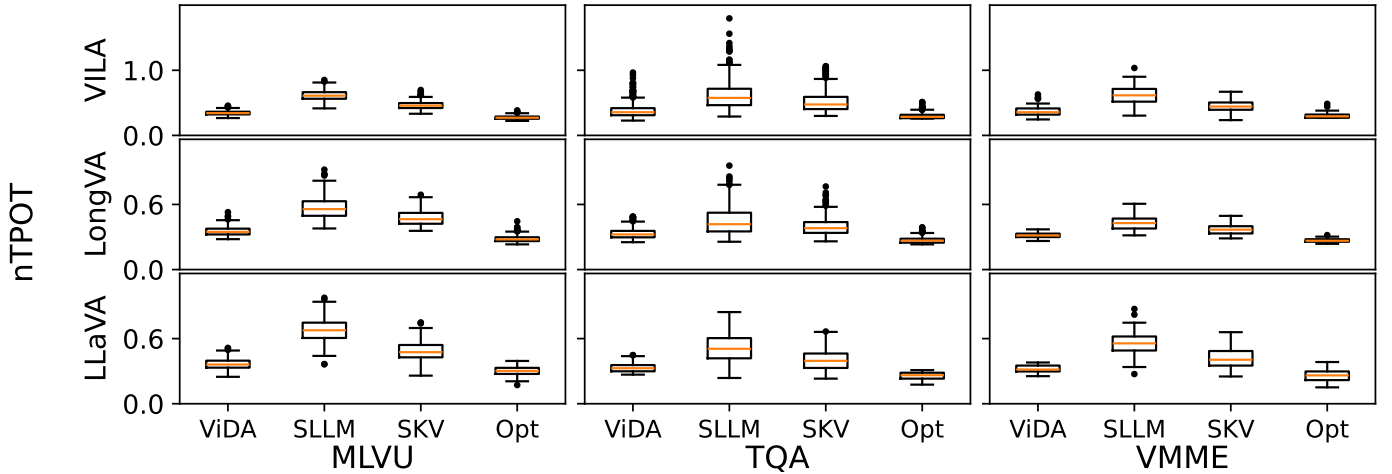


Fig. 13: Comparison results for ViDA against two baselines and the optimal solution under nine VideoQA workloads.

effects of actions depicted in a video, testing the model’s capacity for complex causal reasoning [35].

Combinations of these three VideoQA tasks and three VideoLLMs lead to nine workloads.

Baseline. We use three primary baselines for evaluation: standard autoregressive inference and two MagicDec implementations [21] that employ different sparse methods. These sparse methods are StreamingLLM (SLLM) [30], configured with a token budget of 512, and SnapKV (SKV) [31], configured with a token budget of 2049. These budget values are consistent with those specified in [21]. Besides, we also consider the optimal performance achieved via a brute-force search over the critical token set and budget space.

Key Metrics. We evaluate ViDA and baselines using the normalized TPOT, which is defined as the ratio of measured TPOT over that of autoregressive inference, indicating the VideoLLM’s per-token generation latency. The ideal nTPOT should be as small as possible. Because our approach is built upon speculative decoding, the underlying verification mechanism mathematically guarantees that the output results remain identical to those of the autoregressive inference. Therefore, our detailed metrics focus exclusively on generation speed.

B. Comparison Results

Figure 13 presents the nTPOT of ViDA and each baseline. ViDA consistently outperforms all alternatives in terms of nTPOT. Specifically, the average nTPOT of SLLM, SKV and AR is 1.36 to 1.85, 1.18 to 1.35 and 2.61 to 3.17 times of ViDA’s respectively. To understand the performance differences, we conjecture that ViDA’s advantage arises from two key factors, outlined as follows. We also provide a detailed ablation study in §V-C to quantify the impact of each design choice of ViDA on nTPOT reduction.

First, existing sparse self-speculation, designed for text-based tasks, relies on fixed token budgets, which cannot balance the trade-off between draft latency T_{draft} and acceptance rate α or allocate token budgets tailored to each DV-Step. In contrast, ViDA employs an adaptive strategy designed to

achieve a budget distribution closer to optimal. We use the Jensen-Shannon Divergence (JSD) to quantify this closeness. JSD, a metric ranging from 0 (identical distributions) to 1 (maximally different), indicates smaller distance for closer distributions. Figure 14 shows the JSD between the budget distributions of SKV, SLLM, and ViDA against the optimal distribution. ViDA is quite close to the optimal one with a JSD of 0.05 to 0.22 across the nine workloads. On the contrary, SKV and SLLM have JSD 3.95 to 13.60 and 2.27 to 14.60 times of ViDA’s. Therefore, ViDA’s adaptive token budget selection strategy closely approximates the optimal allocation, leading to a lower nTPOT.

Secondly, existing sparse self-speculation, designed for text-based tasks, struggles to reliably identify critical tokens. This is because their sparse methods often lack awareness of the discrepancy in multimodal tokens, leading to an incorrect set of tokens being selected, which in turn restricts acceptance rate α . In contrast, ViDA utilizes discrepancy-aware token selection, resulting in a proper critical token set. To validate this, we measured the overlap rate between the tokens selected by each method (StreamingLLM, SnapKV, and ViDA) against the optimal set of critical tokens derived by brute-force, under the same token budget of 3k in Figure 15. ViDA’s overlap ratio is 1.13 to 2.03 and 1.84 to 4.80 times of SKV’s and SLLM’s. Therefore, ViDA’s discrepancy-aware token selection strategy accurately identifies critical token set, leading to a higher acceptance rate and eventually a lower nTPOT.

Besides, though the optimal nTPOT derived by brute force is 14.91% to 22.46% lower than ViDA’s, the brute force search has high complexity of 7.64s per DV-Step. Meanwhile, ViDA’s complexity is only 0.02s per DV-Step. Namely, ViDA only requires 0.26% of time to achieve around 80% of the optimal speedup performance. Moreover, the brute-force search is impractical for real-time VideoQA tasks, since it necessitates full attention computation at each iteration, thereby undermining the goal of reducing computation through sparsity.

To determine how the improvements in nTPOT affect overall performance, we evaluated the end-to-end latency, normalized by the highest across all tests, as shown in Figure 16.

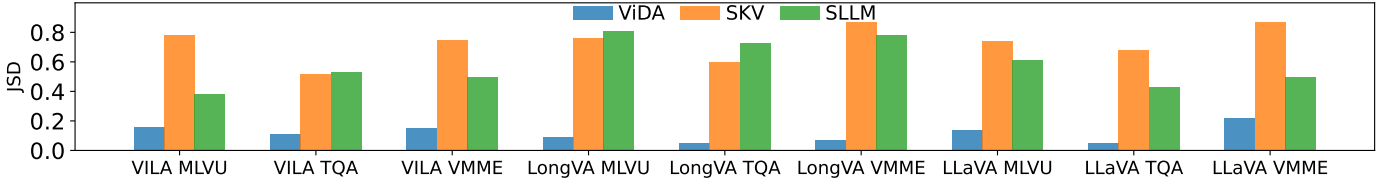


Fig. 14: JSD of token budget distributions for ViDA and two baselines relative to the optimal distribution.

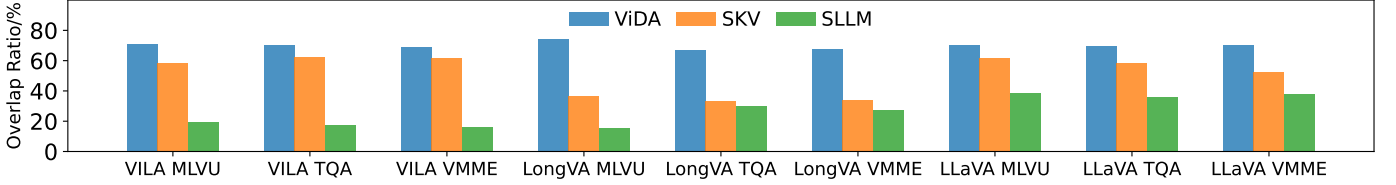


Fig. 15: Overlap ratio of selected token set for ViDA and two baselines relative to the optimal token set.

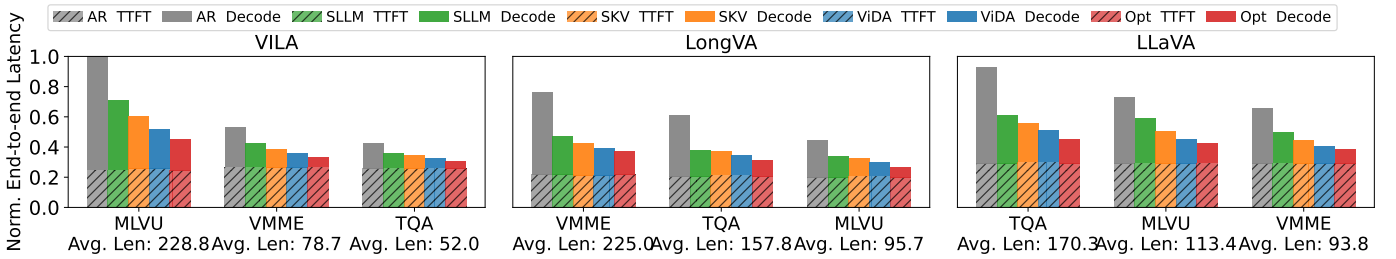


Fig. 16: Normalized average end-to-end latency of ViDA, baselines, and the optimal solution across nine VideoQA workloads with varying average output token lengths. TTFT (Time To First Token) represents the latency before the decoding phase.

The results demonstrate that ViDA consistently achieves the lowest end-to-end latency among all baselines. Specifically, the latency of SLLM, SKV and AR is 1.10 to 1.38, 1.07 to 1.18 and 1.32 to 1.94 times of ViDA's respectively. Furthermore, ViDA performs closely to the optimal, with its latency being only 5.1% to 13.5% higher.

This performance advantage is particularly pronounced as the average output length grows, because ViDA's optimizations focus on minimizing TPOT, which directly represents the per-token decoding latency. As longer outputs increase the cumulative decoding latency, the benefit of the low TPOT achieved by ViDA is significantly amplified in the overall end-to-end latency.

C. Ablation Study

In this section, we disable each of ViDA's novel features to demonstrate its contribution to guaranteeing low nTPOT. Due to space limitations, we only present results for the MLVU dataset, as performance on the other datasets is similar.

How important is adaptive budget allocation? ViDA uses an adaptive token budget allocation strategy to reduce nTPOT. To evaluate its effectiveness, we compare ViDA with: (1) ViDA-Fb (Fixed budget), employing eight fixed budget configurations (1k to 8k in increments of 1k); (2) ViDA-Es (Exhaustive search), a variant that finds the budget by exhaustively searching the entire budget space instead of using progressive search; (3) ViDA-Wbt (Without budget tracker), a variant that uses the budget value at which progressive search

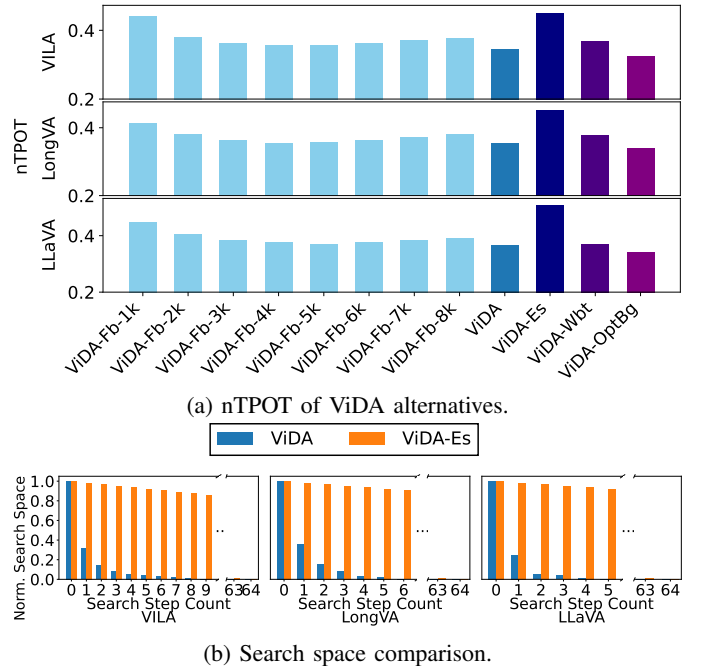


Fig. 17: Ablation results of adaptive budget allocation.

converges; and (4) ViDA-OptBg (Optimal budget), the optimal budget allocation found through brute-force search.

As shown in Figure 17a, adaptive budget allocation in ViDA achieves the lowest nTPOT among ViDA-Fb, ViDA-Es, and

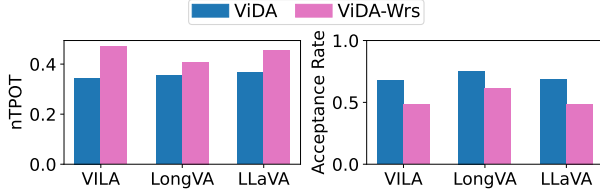


Fig. 18: Ablation results of re-selection.

ViDA-Wbt. It reduces nTPOT by up to 21.90% and 6.76% compared to ViDA-Fb and ViDA-Wbt, demonstrating that adaptive allocation alongside token budget tracker effectively contributes to inference acceleration.

Figure 17b shows the average search space size for ViDA and ViDA-Es when searching converges as search step counts. ViDA converges rapidly, within an average of 4.21 to 6.89 steps, while ViDA-Es requires traversing the full search space, taking 64 steps. This progressive search proposed by ViDA effectively reduces the search space without performance sacrifice in terms of token budget⁵, resulting in a 91.47% reduction in average search steps compared to ViDA-Es. This faster convergence helps ViDA achieve up to a 27.57% lower nTPOT compared to ViDA-Es, avoiding performance degradation due to prolonged searching.

Besides, ViDA achieves 94.61% of ViDA-OptBg’s performance, demonstrating that its adaptive allocation determines token budgets closely approaching the optimum. Moreover, ViDA can be used in real-time VideoQA applications, while ViDA-OptBg cannot because it requires a brute-force search across all possible token budgets.

How important is re-selection? ViDA employs a critical token reuse mechanism that reuses the token indices selected during the prefill phase and re-selects them during the verification phase. To assess the effectiveness of re-selection, we compare ViDA with ViDA-Wrs (ViDA without re-selection). As shown in Figure 18, ViDA achieves 13.15% to 26.81% lower nTPOT than ViDA-WRS, which verifies the effectiveness of re-selection on nTPOT reduction.

To further understand the performance improvements enabled by re-selection, we analyze its impact based on Equation 1. The re-selection mechanism affects both the acceptance rate (α) and the verification latency (T_{verify}). By re-selecting during the verification phase, ViDA maintains the selection of reliable critical tokens as iterations increase, resulting in a higher α . However, this re-selection introduces additional computational overhead, increasing T_{verify} . As illustrated in Figure 18, the re-selection mechanism improves α by 23.55% to 40.57%, a gain sufficient to offset the overhead and achieve an overall speedup improvement.

How important is layer-wise selection? ViDA captures the varying distributions of critical tokens across different layers by selecting them individually at each layer. To verify the effectiveness of layer-wise selection, we compare ViDA with ViDA-Wls (ViDA without layer-wise selection), which selects critical tokens in a shallow layer, similar to FastV [14].

⁵Theorem 1 and candidate budget tracker guarantee that ViDA and ViDA-Es will have the same output token budget.

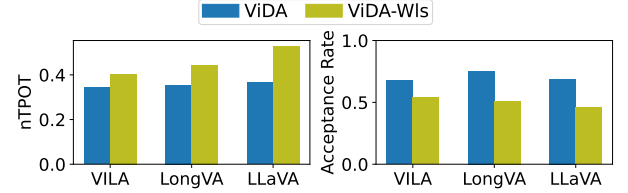
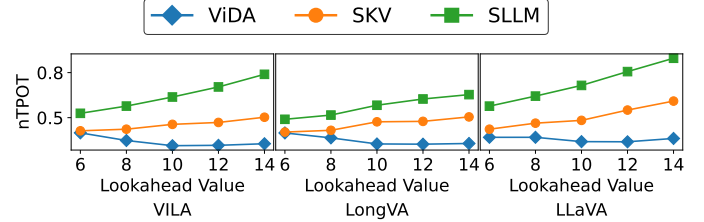


Fig. 19: Ablation results of layer-wise selection.

Fig. 20: Sensitivity analysis of lookahead k .

As shown in Figure 19, ViDA achieves 14.56% to 30.42% lower nTPOT than ViDA-WLS. The reason for this improvement is similar to that of the re-selection mechanism’s performance enhancement. Selecting layer by layer simultaneously increases α and T_{verify} . As illustrated in Figure 19, α improves by 25.69% to 50.30%, a gain sufficient to offset the additional computational overhead and thereby enhance overall performance.

D. Sensitivity and Overhead Analysis

Sensitivity analysis of k . For lookahead $k = 10$, ViDA exhibits superior performance compared with SLLM and SKV. To evaluate its performance across varying k , we assess ViDA, SLLM, and SKV with k values of [6, 8, 10, 12, 14].

As illustrated in Figure 20, ViDA consistently outperforms SLLM and SKV across all benchmarks, achieving up to 53.63% lower nTPOT than SLLM and up to 35.02% lower nTPOT than SKV. Notably, the trend of nTPOT with respect to varying k differs between ViDA and the other two. For SLLM and SKV, nTPOT gradually increases as k increases. This suggests that their acceptance rate plateaus at lower k . Increasing k beyond this point does not enhance the number of accepted draft tokens and instead increases nTPOT due to the increased number of draft attempts.

In contrast, ViDA’s nTPOT initially decreases as k increases and then stabilizes. This behavior is attributed to ViDA’s discrepancy-aware token selection, which continues to accept more draft tokens even at higher k , thereby elevating the upper limit of acceleration performance. Consequently, ViDA maintains low nTPOT across different k .

Sensitivity analysis of partition granularity n . In §IV-B, the context length is split into n blocks to form a discrete token budget search space. To evaluate the sensitivity of n , we re-exam n from [16, 32, 64, 96, 128].

As shown in Figure 21, 64 is nearly the optimal choice, while nTPOT first decreases and then increases as n increases, with a maximum performance fluctuation of 10.46%. This occurs because when n is small, the search converges faster,

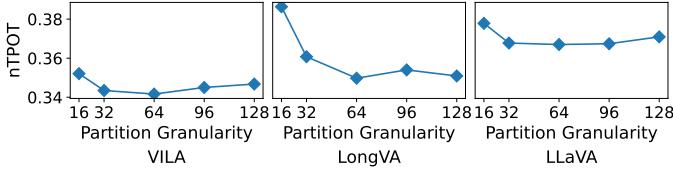


Fig. 21: Sensitivity analysis of partition granularity n .

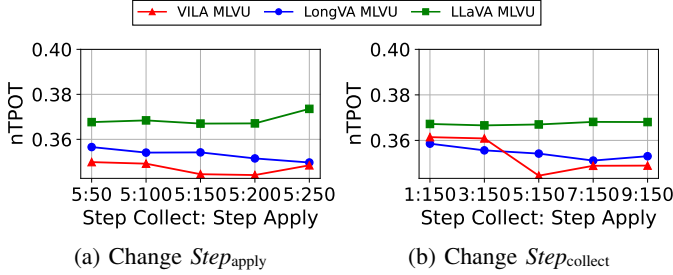


Fig. 22: Sensitivity analysis of $Step_{collect}:Step_{apply}$.

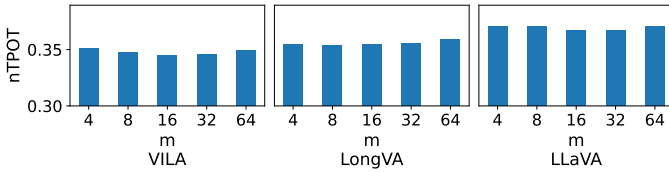


Fig. 23: Sensitivity analysis of m .

but the search space contains too few token budget candidates, leading to the selection of suboptimal budgets while better options are excluded. Conversely, when n is too large, the search may converge more slowly, resulting in performance loss during the convergence process. Choosing an appropriate n ensures both the speed of convergence and the selection of appropriate budgets, thereby performing best.

Sensitivity to $Step_{collect}:Step_{apply}$. ViDA uses two parameters, $Step_{collect}$ and $Step_{apply}$, in §IV-B to control the number of DV-Steps during budget searches and the number of DV-Steps applied after identifying the proper token budget. To evaluate the sensitivity of $Step_{collect}:Step_{apply}$, we test different settings by varying both $Step_{collect}$ and $Step_{apply}$ within two groups: 5:[50, 100, 150, 200, 250] and [1, 3, 5, 7, 9]:150.

As shown in Figure 22a, these settings have minimal effects on nTPOT, with average variations ranging from 1.65% to 1.92%. The variation in Figure 22b is also minor, ranging from 0.42% to 4.68%. These results demonstrate the robustness of the progressive search to $Step_{collect}:Step_{apply}$. The 5:150 configuration performs well in most tests. This is because a small $Step_{collect}:Step_{apply}$ ratio leads to inaccurate estimations of the unknown α_{lb} for each B_{lb} , while a larger ratio results in prolonged application of a given token budget. Therefore, choosing a balanced ratio like 5:150 is effective.

Sensitivity analysis of m . In §IV-C, ViDA sums the last m rows of attention weight to get an importance vector. To evaluate ViDA’s sensitivity to parameter m , we tested m values of [4, 8, 16, 32, 64] and measured their impact on nTPOT. As illustrated in Figure 23, nTPOT varies little with m , showing a maximum difference of only 1.95%, demon-

strating ViDA’s robustness to this parameter. An intermediate value like $m = 16$ performs well.

This performance trend occurs because attention weight of the most recent tokens are often crucial for determining current token importance. When m is small, the importance measurement relies excessively only on very recent tokens and is not informative. Conversely, when m is large, including too many tokens dilutes this crucial recent information. Therefore, choosing $m = 16$ is effective.

Overhead Analysis. The computational overhead in ViDA primarily consists of two parts: selecting the critical visual tokens, which accounts for approximately 4% of the extra overhead, and the progressive search for the token budget. Because the progressive search only involves basic scalar comparisons and assignments, its time and space complexities are $O(1)$, resulting in a negligible 0.1% empirical overhead. Aside from computation, the memory footprint remains minimal, requiring less than 2.44 MB to store the critical indices.

E. Compatibility with FlashDecoding

The original implementation of ViDA utilizes standard FlashAttention as the backend. As scaling attention computation along the sequence dimension becomes increasingly crucial for long-context generation, we further evaluated ViDA’s compatibility with FlashDecoding to demonstrate its robustness across different attention kernels. As shown in Figure 24, the nTPOT of SLLM, SKV, and AR is 1.43 to 2.05, 1.16 to 1.48, and 2.16 to 2.76 times that of ViDA, respectively.

Notably, the inflection point shifts up to approximately 8k tokens because FlashDecoding improves hardware utilization. In this scenario, ViDA’s adaptive token budget allocator leverages this extended inflection point as a higher initial lower bound, effectively reducing the remaining search space. Furthermore, since the static budgets of SLLM and SKV fall significantly further behind this newly elevated lower bound, ViDA’s adaptive allocation widens its performance margin, achieving even higher maximum speedup ratios compared with these baselines.

F. Adaptation and Comparison with Visual Sparse Methods

Visual token compression and pruning methods (e.g., PVC [26], ST3 [25], TokenPacker [22] and LOOK-M [16]) reduce computational overhead by permanently condensing or discarding visual tokens. Direct comparison with ViDA is infeasible as these approaches are fundamentally lossy and alter the generated output, whereas ViDA preserves the original model output. Furthermore, while ViDA is a training-free and plug-and-play framework, approaches like TokenPacker and PVC require extensive model retraining or fine-tuning.

Nevertheless, to isolate and rigorously evaluate the efficacy of their core sparsity mechanisms (encompassing both token selection criteria and budget allocation), we adapted the two training-free methods, LOOK-M and ST3, into sparse self-speculation framework, denoting them as SLOOKM and SST3. Figure 25 shows the nTPOT of SLOOKM and SST3 is 1.27 to 2.23 and 1.59 to 1.95 times that of ViDA, respectively. This is because they fail to achieve a favorable trade-off between T_{draft} and α .

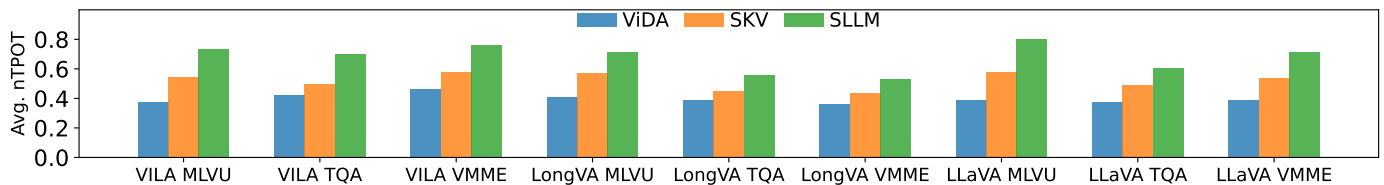


Fig. 24: nTPOT of ViDA and baselines with FlashDecoding.

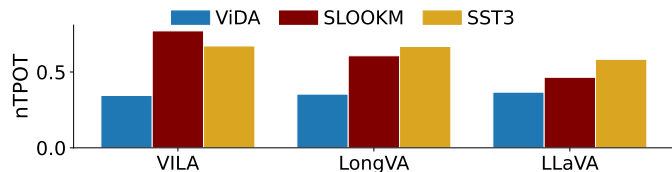


Fig. 25: nTPOT of ViDA, SLOOKM and SST3 under MLVU.

G. Robustness Across Hardware Architectures

To evaluate whether ViDA’s heuristics and parameters remain robust across diverse hardware architectures, we extended our experiments to a server equipped with an NVIDIA H800 GPU.

As shown in Figure 26, ViDA consistently maintains its performance advantages in end-to-end generation. Under the new hardware constraints, the nTPOT of the SLLM, SKV and AR is 1.40 to 1.67, 1.15 to 1.26, and 2.27 to 2.98 times of ViDA’s respectively, demonstrating that our acceleration effectiveness is not limited to a specific GPU architecture.

Furthermore, we replicated the ablation studies and sensitivity analyses on the H800 GPU. The results, illustrated in Figure 27 and Figure 28, exhibit the exact same performance trends as those observed on the A800. Specifically, our core heuristics, including adaptive budget allocation, layer-wise selection, and re-selection, consistently contribute to significant performance improvements. Concurrently, the optimal hyperparameter configurations remain stable on the new hardware, proving that our overall system design is highly robust.

This cross-architecture robustness is fundamentally attributed to two key aspects of our system design. First, the visual sparse patterns we observed are inherent to the models and workloads, making them completely hardware-agnostic. Consequently, the discrepancy-aware token selection grounded in these universal patterns naturally maintains its effectiveness across different architectures. Second, our adaptive budget allocator guarantees robustness by combining offline profiling with online allocation. We first locate the optimal budget inflection point by profiling the hardware characteristics offline, and then the progressive search dynamically allocates the token budget on the fly during online inference.

VI. RELATED WORK

Token Pruning. Many inference systems use token pruning to reduce latency. SnapKV [31], H2O [36] and StreamingLLM [30] exploit the sparsity of textual tokens for pruning. MagicPig [37] and Quest [38] identify critical tokens via hashing and dimensionality reduction. These methods

accelerate text generation. Furthermore, FastV [14], LOOKM [16] and Llava-Pruner [39] extend pruning to multi-modal scenarios, using visual token sparsity to accelerate tasks like VideoQA and image-text dialogue. While pruning reduces token count to cut computational costs, it may compromise output quality, failing to meet correctness requirements.

Speculative Decoding. Several inference systems utilize small draft models for speculative decoding. For example, Triforce [40] employs a hierarchical structure for speculation, using the lowest-level model to draft. MLLMSpec [27] uses a fine-tuned multimodal model to draft, while Cascaded-Spec [41] relies on cascaded small models. These methods achieve lossless acceleration, but the inference efficiency heavily depends on the quality of the draft output. Finding suitable native small models for different tasks is difficult, so most systems use fine-tuning, which limits generalization and acceptance rate, which may not meet the usability requirements. **Native Sparse Attention Design.** Other systems boost inference efficiency by modifying the attention mechanism to replace full attention (on all Key-Value pairs) with sparse attention computing only a KV subset. Unlike token pruning, this design, such as DSA [42], NSA [43], [44], and MoBA [45], fundamentally requires architectural redesign and full retraining. Thereby, the effectiveness of these natively sparse models heavily depends on the training data and their generalization across diverse tasks remains a challenge [42]. Meanwhile, ViDA achieves lossless acceleration in a training-free manner.

Inference Optimizations. In addition, many systems propose other optimization techniques for LLM inference, such as operator fusion [46], [47], request scheduling [48], [49], quantization [50], [51], and parallelism [52], [53]. ViDA’s selective sparse self-speculation is orthogonal to these techniques and can be combined to accelerate inference further.

VII. CONCLUSION

In this paper, we introduce ViDA, an inference system designed to accelerate VideoQA tasks by dynamically selecting critical tokens on a flexible budget for selective sparse self-speculation. ViDA incorporates innovative adaptive budget allocation and discrepancy-aware token selection techniques to decide both the number of tokens to select and which tokens to prioritize. Unlike existing uniform sparse self-speculation, which often suffers from incorrect token selection or misallocation, ViDA’s approach offers a significant improvement in inference speedup. Evaluation shows that compared to state-of-the-art methods, ViDA preserves exact model outputs while reducing average time-per-output-token by 15% to 46% and average end-to-end latency by up to 28%, while decreasing

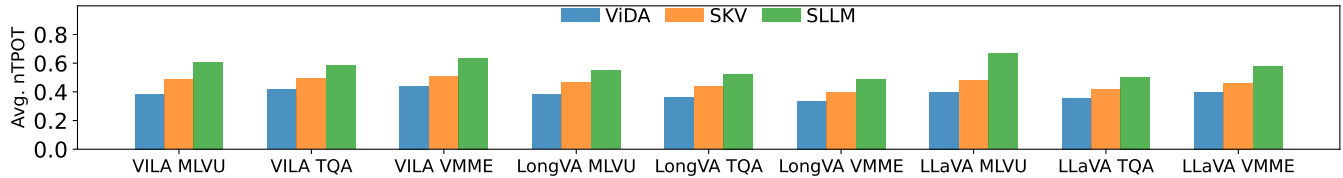


Fig. 26: nTPOT of ViDA and baselines in H800 GPU.

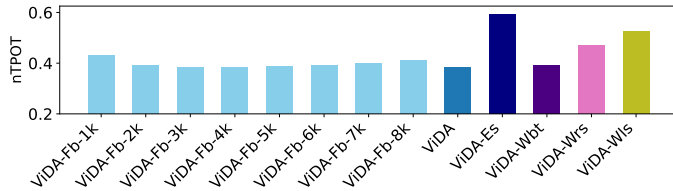


Fig. 27: Ablation study on LongVA-MLVU workload in H800.

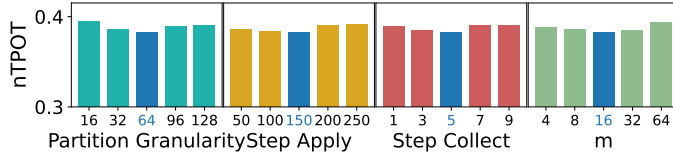


Fig. 28: Sensitivity analysis on LongVA-MLVU workload in H800.

the divergence from optimal token budget distribution by up to 93% and increasing the optimal critical token ratio by up to 380%.

REFERENCES

- [1] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," *Advances in neural information processing systems*, vol. 36, 2024.
- [2] D. Zhu, J. Chen, X. Shen, X. Li, and M. Elhoseiny, "Minigt-4: Enhancing vision-language understanding with advanced large language models," *arXiv preprint arXiv:2304.10592*, 2023.
- [3] J. Chen, D. Zhu, X. Shen, X. Li, Z. Liu, P. Zhang, R. Krishnamoorthi, V. Chandra, Y. Xiong, and M. Elhoseiny, "Minigt-v2: large language model as a unified interface for vision-language multi-task learning," *arXiv preprint arXiv:2310.09478*, 2023.
- [4] P. Zhang, K. Zhang, B. Li, G. Zeng, J. Yang, Y. Zhang, Z. Wang, H. Tan, C. Li, and Z. Liu, "Long context transfer from language to vision," *arXiv preprint arXiv:2406.16852*, 2024.
- [5] F. Xue, Y. Chen, D. Li, Q. Hu, L. Zhu, X. Li, Y. Fang, H. Tang, S. Yang, Z. Liu *et al.*, "Longvila: Scaling long-context visual language models for long videos," *arXiv preprint arXiv:2408.10188*, 2024.
- [6] Y. Tang, J. Bi, S. Xu, L. Song, S. Liang, T. Wang, D. Zhang, J. An, J. Lin, R. Zhu *et al.*, "Video understanding with large language models: A survey," *arXiv preprint arXiv:2312.17432*, 2023.
- [7] J. Chen, Z. Lv, S. Wu, K. Q. Lin, C. Song, D. Gao, J.-W. Liu, Z. Gao, D. Mao, and M. Z. Shou, "Videollm-online: Online video large language model for streaming video," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 18 407–18 418.
- [8] C. Fu, H. Lin, Z. Long, Y. Shen, M. Zhao, Y. Zhang, S. Dong, X. Wang, D. Yin, L. Ma *et al.*, "Vita: Towards open-source interactive omni multimodal llm," *arXiv preprint arXiv:2408.05211*, 2024.
- [9] R. Zhu, Z. Jiang, Z. Zhang, X. Liu, X. Liu, and X. Jin, "Cannikin: No lagging of slo in concurrent multiple lora llm serving," *IEEE Transactions on Parallel and Distributed Systems*, vol. 36, no. 9, pp. 1972–1984, 2025.
- [10] Z. Lin, Y. Miao, G. Xu, C. Li, O. Saarikivi, S. Maleki, and F. Yang, "Efficient schedule construction for distributed execution of large dnn models," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 12, pp. 2375–2391, 2024.
- [11] C. A. Navarro, R. Carrasco, R. J. Barrientos, J. A. Riquelme, and R. Vega, "Gpu tensor cores for fast arithmetic reductions," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 1, pp. 72–84, 2021.
- [12] T. Faingnaert, T. Besard, and B. De Sutter, "Flexible performant gemm kernels on gpus," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 9, pp. 2230–2248, 2022.
- [13] Z. Xue, M. Wen, J. Yang, M. Tang, Z. Luo, J. Feng, Y. Shi, Z. Chen, J. Shen, and J. Langguth, "Sspmm: Efficiently scalable spmm kernels across multiple generations of tensor cores," *IEEE Transactions on Parallel and Distributed Systems*, pp. 1–17, 2025.
- [14] L. Chen, H. Zhao, T. Liu, S. Bai, J. Lin, C. Zhou, and B. Chang, "An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models," in *European Conference on Computer Vision*. Springer, 2024, pp. 19–35.
- [15] W. Ye, Q. Wu, W. Lin, and Y. Zhou, "Fit and prune: Fast and training-free visual token pruning for multi-modal large language models," *arXiv preprint arXiv:2409.10197*, 2024.
- [16] Z. Wan, Z. Wu, C. Liu, J. Huang, Z. Zhu, P. Jin, L. Wang, and L. Yuan, "Look-m: Look-once optimization in kv cache for efficient multimodal long-context inference," *arXiv preprint arXiv:2406.18139*, 2024.
- [17] Y. Zhong, Z. Liu, Y. Li, and L. Wang, "Aim: Adaptive inference of multi-modal llms via token merging and pruning," *arXiv preprint arXiv:2412.03248*, 2024.
- [18] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi *et al.*, "Specinfer: Accelerating large language model serving with tree-based speculative inference and verification," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 932–949.
- [19] B. Spector and C. Re, "Accelerating llm inference with staged speculative decoding," *arXiv preprint arXiv:2308.04623*, 2023.
- [20] M. Yan, S. Agarwal, and S. Venkataraman, "Decoding speculative decoding," *arXiv preprint arXiv:2402.01528*, 2024.
- [21] J. Chen, V. Tiwari, R. Sadhukhan, Z. Chen, J. Shi, I. E.-H. Yen, and B. Chen, "Magicdec: Breaking the latency-throughput tradeoff for long context generation with speculative decoding," *arXiv preprint arXiv:2408.11049*, 2024.
- [22] W. Li, Y. Yuan, J. Liu, D. Tang, S. Wang, J. Qin, J. Zhu, and L. Zhang, "Tokenpacker: Efficient visual projector for multimodal llm," *arXiv preprint arXiv:2407.02392*, 2024.
- [23] Q. Wu, W. Lin, W. Ye, Y. Zhou, X. Sun, and R. Ji, "Accelerating multimodal large language models via dynamic visual-token exit and the empirical findings," *arXiv preprint arXiv:2411.19628*, 2024.
- [24] Z. Zhang, P. Pham, W. Zhao, K. Wan, Y.-J. Li, J. Zhou, D. Miranda, A. Kale, and C. Xu, "Treat visual tokens as text? but your mllm only needs fewer efforts to see," *arXiv preprint arXiv:2410.06169*, 2024.
- [25] J. Zhuang, L. Lu, M. Dai, R. Hu, J. Chen, Q. Liu, and H. Hu, "St³: Accelerating multimodal large language model by spatial-temporal visual token trimming," *arXiv preprint arXiv:2412.20105*, 2024.
- [26] C. Yang, X. Dong, X. Zhu, W. Su, J. Wang, H. Tian, Z. Chen, W. Wang, L. Lu, and J. Dai, "Pvc: Progressive visual token compression for unified image and video processing in large vision-language models," *arXiv preprint arXiv:2412.09613*, 2024.
- [27] M. Gagrani, R. Goel, W. Jeon, J. Park, M. Lee, and C. Lott, "On speculative decoding for multimodal large language models," *arXiv preprint arXiv:2404.08856*, 2024.
- [28] Z. He, Z. Zhong, T. Cai, J. D. Lee, and D. He, "Rest: Retrieval-based speculative decoding," *arXiv preprint arXiv:2311.08252*, 2023.
- [29] L. Zhang, X. Wang, Y. Huang, and R. Xu, "Learning harmonized representations for speculative sampling," *arXiv preprint arXiv:2408.15766*, 2024.

- [30] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," *arXiv preprint arXiv:2309.17453*, 2023.
- [31] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, "Snapkv: Llm knows what you are looking for before generation," *arXiv preprint arXiv:2404.14469*, 2024.
- [32] X. An, Y. Xie, K. Yang, W. Zhang, X. Zhao, Z. Cheng, Y. Wang, S. Xu, C. Chen, D. Zhu *et al.*, "Llava-onevision-1.5: Fully open framework for democratized multimodal training," *arXiv preprint arXiv:2509.23661*, 2025.
- [33] J. Zhou, Y. Shu, B. Zhao, B. Wu, S. Xiao, X. Yang, Y. Xiong, B. Zhang, T. Huang, and Z. Liu, "Mlvu: A comprehensive benchmark for multi-task long video understanding," *arXiv preprint arXiv:2406.04264*, 2024.
- [34] M. Maaz, H. Rasheed, S. Khan, and F. S. Khan, "Video-chatgpt: Towards detailed video understanding via large vision and language models," *arXiv preprint arXiv:2306.05424*, 2023.
- [35] C. Fu, Y. Dai, Y. Luo, L. Li, S. Ren, R. Zhang, Z. Wang, C. Zhou, Y. Shen, M. Zhang *et al.*, "Video-mme: The first-ever comprehensive evaluation benchmark of multi-modal llms in video analysis," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2025, pp. 24 108–24 118.
- [36] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett *et al.*, "H2o: Heavy-hitter oracle for efficient generative inference of large language models," *Advances in Neural Information Processing Systems*, vol. 36, pp. 34 661–34 710, 2023.
- [37] Z. Chen, R. Sadhukhan, Z. Ye, Y. Zhou, J. Zhang, N. Nolte, Y. Tian, M. Douze, L. Bottou, Z. Jia *et al.*, "Magicpig: Lsh sampling for efficient llm generation," *arXiv preprint arXiv:2410.16179*, 2024.
- [38] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han, "Quest: Query-aware sparsity for efficient long-context llm inference," *arXiv preprint arXiv:2406.10774*, 2024.
- [39] Y. Shang, M. Cai, B. Xu, Y. J. Lee, and Y. Yan, "Llava-prumerge: Adaptive token reduction for efficient large multimodal models," *arXiv preprint arXiv:2403.15388*, 2024.
- [40] H. Sun, Z. Chen, X. Yang, Y. Tian, and B. Chen, "Triforce: Lossless acceleration of long sequence generation with hierarchical speculative decoding," *arXiv preprint arXiv:2404.11912*, 2024.
- [41] Z. Chen, X. Yang, J. Lin, C. Sun, K. C.-C. Chang, and J. Huang, "Cascade speculative drafting for even faster llm inference," *arXiv preprint arXiv:2312.11462*, 2023.
- [42] DeepSeek-AI, "Deepseek-v3.2-exp: Boosting long-context efficiency with deepseek sparse attention," 2025.
- [43] J. Yuan, H. Gao, D. Dai, J. Luo, L. Zhao, Z. Zhang, Z. Xie, Y. Wei, L. Wang, Z. Xiao *et al.*, "Native sparse attention: Hardware-aligned and natively trainable sparse attention," *arXiv preprint arXiv:2502.11089*, 2025.
- [44] E. Song, W. Chai, S. Yang, E. Armand, X. Shan, H. Xu, J. Xie, and Z. Tu, "Videosa: Native sparse attention scales video understanding," *arXiv preprint arXiv:2510.02295*, 2025.
- [45] E. Lu, Z. Jiang, J. Liu, Y. Du, T. Jiang, C. Hong, S. Liu, W. He, E. Yuan, Y. Wang *et al.*, "Moba: Mixture of block attention for long-context llms," *arXiv preprint arXiv:2502.13189*, 2025.
- [46] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," *Advances in Neural Information Processing Systems*, vol. 35, pp. 16 344–16 359, 2022.
- [47] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," *arXiv preprint arXiv:2307.08691*, 2023.
- [48] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for {Transformer-Based} generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 521–538.
- [49] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 611–626.
- [50] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2022.
- [51] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for on-device llm compression and acceleration," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 87–100, 2024.
- [52] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," *Proceedings of Machine Learning and Systems*, vol. 5, pp. 606–624, 2023.
- [53] B. Wu, S. Liu, Y. Zhong, P. Sun, X. Liu, and X. Jin, "Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism," in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, 2024, pp. 640–654.



Liang Zheng received the B.E. degree in data science and big data technology from Northwest A&F University, Shaanxi, China, in 2024. He is currently working toward the Ph.D. degree in computer science and technology from Tianjin University, Tianjin, China. His research interests include machine learning systems and cloud computing. In particular, in machine learning systems, his research focuses on inference and deployment optimization for LLMs.



Yitao Hu received the B.S. degree from Shanghai Jiao Tong University in 2014, and the Ph.D. degree from University of Southern California in 2021. His research interests span machine learning systems, high-performance computer systems, and hardware-software co-design, and has published over 30 papers in leading conferences and journals, including IEEE Transactions on Parallel and Distributed Systems, IEEE Transactions on Computers, ASPLOS, EuroSys, SoCC, INFOCOM, IWQoS, and SIGCOMM. He served as the Associate Editor for IEEE Transactions on Computers, and received the Best Paper Award at ACM SoCC 2024.



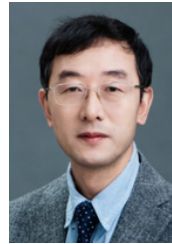
Chen Shen received the B.E. degree in computer science and technology from Dalian University of Technology, Dalian, Liaoning, China, in 2023. He is currently working toward the master's degree in computer science and technology with Tianjin University, Tianjin, China. His research direction is network and cloud computing, focusing on efficient inference acceleration, and deployment of large language models.



Zhixin Zhao (Graduate Student Member, IEEE) received the B.E. degree in computer science and technology from the Northwest A&F University, Shaanxi, China, in 2022. He is currently working toward the Ph.D. degree in software engineering from Tianjin University, Tianjin, China. His research interests include machine learning systems and cloud computing. In particular, in machine learning systems, his research focuses on inference and deployment optimization for DNN models and LLMs.



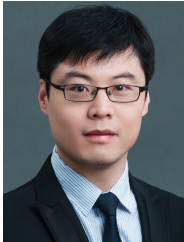
Bowen Shi received the B.E. degree in computer science and technology from the Northwest A&F University, Shaanxi, China, in 2025. He is currently working toward the master's degree in computer science and technology with Tianjin University, Tianjin, China. His research interests include machine learning systems and cloud computing. In particular, in machine learning systems, his research focuses on inference and deployment optimization for LLMs.



Keqiu Li (Fellow, IEEE) received the bachelor's and master's degrees from the Department of Applied Mathematics, Dalian University of Technology, in 1994 and 1997, respectively, and the PhD degree from the Graduate School of Information Science, Japan Advanced Institute of Science and Technology, in 2005. He is currently a full professor, the dean of the College of Intelligence and Computing, Tianjin University, China. He is the recipient of National Science Foundation for Distinguished Young Scholars of China. He keeps working on the topics of blockchain, mobile computing, datacenter, and cloud computing. He has more than 150 papers published on prestigious journals or conferences such as IEEE/ACM Transactions on Networking, IEEE Transactions on Parallel and Distributed Systems, IEEE Transactions on Computers, IEEE Transactions on Mobile Computing, INFOCOM, and ICNP.



Yuhao Zhang received the Ph.D. degree from the School of Computer Science and Technology, Shandong University, Qingdao, China, in June 2022. His research interests include computer architecture, storage system, embedded system, and processing-in-memory.



Laiping Zhao (Member, IEEE) received the B.S. and M.S. degrees from the Dalian University of Technology, Dalian, China, in 2007 and 2009, respectively, and the Ph.D. degree from the Department of Informatics, Kyushu University, Fukuoka, Japan, in 2012. He is currently a Professor with the College of Intelligence and Computing, Tianjin University, Tianjin, China. His research interests include cloud computing and serverless computing. He has authored or co-authored more than 70 papers in international journals and conferences, including IEEE

TRANSACTIONS ON CLOUD COMPUTING, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, TRANSACTIONS ON SERVICE COMPUTING, ASPLOS, SC, ATC, and EuroSys.



Wenxin Li received the B.E. and Ph.D. degrees from the School of Computer Science and Technology, Dalian University of Technology, in 2012 and 2018, respectively. He is currently a Tenured Associate Professor with the College of Intelligence and Computing, Tianjin University, China. He also worked as a Research Assistant with the National University of Defense Technology, China, as a Visiting Student with the Department of Electrical and Computer Engineering, University of Toronto, Canada, as a Postdoctoral Fellow with The Hong Kong University

of Science and Technology, Hong Kong, and as a Researcher with the Theory Laboratory of Huawei, Hong Kong. His research interests include datacenter networks and cloud computing. His research papers were published in many prestigious journals and conferences, including TON, TPDS, INFOCOM, ICDCS, and ICPP. He was a recipient of CCF Outstanding Doctoral Dissertation Award 2020.